

Talk.M3D

Documentação Técnica

Arquitetura, servidor, métodos da API, estrutura de desenvolvimento, banco de dados, implantação e operação.

Sumário

1	Visão geral e arquitetura	2
2	Estrutura do repositório	4
3	Inicialização, configuração e instalação	6
4	Servidor: camadas e ciclo de requisição	8
5	Segurança, autenticação e auditoria	10
6	Domínios funcionais e regras de negócio	12
7	Referência da API	15
8	Banco de dados	20
9	Cliente (PWA)	30
10	Implantação e operação	31
11	Testes, qualidade e CI/CD	33
12	Guia de desenvolvimento	34
13	Limites conhecidos e glossário	35

> **Público:** quem desenvolve, instala, opera e mantém o Talk.M3D. Para o uso diário, consulte o **Manual do Usuário Ilustrado**. Os trechos de código e as tabelas foram extraídos do repositório (versão do código atual).

Convenções

Notação	Significado
`texto monoespaçado`	Arquivo, rota, variável, comando ou trecho de código
`:id`	Parâmetro de caminho
Acesso “sessão”	Qualquer usuário autenticado
Acesso “perfil: X”	Autenticado e com um dos perfis X (ver capítulo 5)
Acesso “pública”	Sem login; protegida por outro mecanismo (token, código ou limite por IP)

1 Visão geral e arquitetura

O que é, do que é feito e como as partes conversam

O Talk.M3D é um **mensageiro clínico** com agenda, teleconsulta, prontuário com IA de apoio, receitas com QR, rotinas de exercício, financeiro, créditos/planos e auditoria. O servidor é uma **aplicação Node.js única** (API REST + WebSocket + arquivos estáticos); o cliente é uma **PWA em JavaScript puro**, sem framework e sem scripts inline.

Camada	Tecnologia	Observações
Servidor	Node.js 22, Fastify 5 (ES modules)	Plugins: helmet, cookie, jwt, rate-limit, websocket, multipart, static
Validação	Zod	Todo corpo, query e parâmetro é validado; erro 400 padronizado
Senhas	Argon2	Mínimo de 12 caracteres (política em `src/policy.js`)
Banco	PostgreSQL (`pg`)	Migrações SQL numeradas; pool de 10 conexões; TLS por padrão
Cifragem	AES-256-GCM (`node:crypto`)	Chave `DATA_KEY` (32 bytes base64); colunas `*_enc`
Vídeo	Agora (`agora-token` no servidor, `agora-rtc-sdk-ng` no cliente)	Token RTC de 2 h por consulta
IA	Gemini, OpenAI, Anthropic (failsafe) e STT OpenAI/Gemini	Chamadas por `fetch`; sem SDKs
Pagamento	PayPal (REST)	Valores sempre calculados no servidor
SMS	Twilio (REST)	Lembretes de consulta
PDF e QR	`pdfkit`, `qrcode`	Documentos e QR de receitas/rotinas
Borda	Nginx ou Caddy	HTTPS, WebSocket e `X-Forwarded-Proto`
Cliente	HTML + JS + CSS, service worker	Instalável (manifest), cache versionado

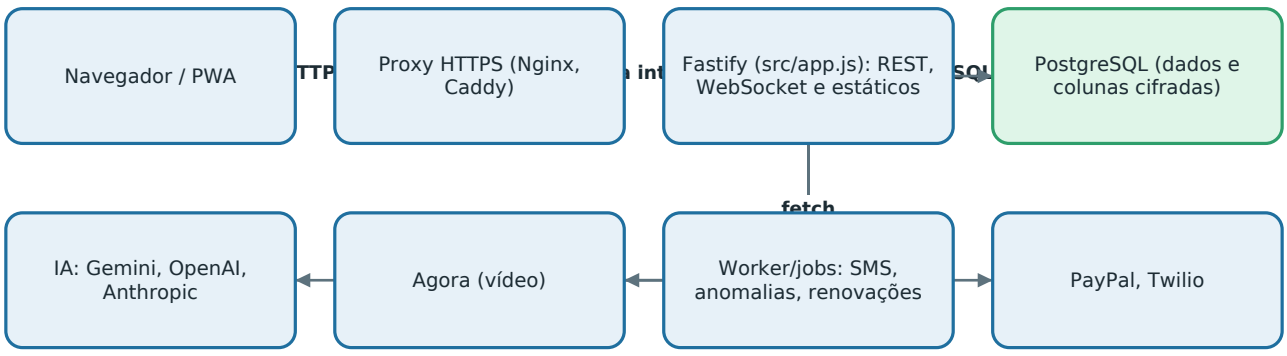


Figura 1.1 · Componentes e fluxo. A API e o worker podem rodar no mesmo processo ou separados (`RUN_JOBS=false` + `src/worker.js`).

Princípios de projeto

- **Instalação sem arquivo .env**: um servidor mínimo de boot guia a configuração pelo navegador (capítulo 3).
- **Dados clínicos cifrados em repouso** e sempre filtrados por vínculo (`care_team`).
- **Auditoria encadeada por hash** e imutável por gatilho do banco.
- **IA como apoio**: rascunhos que exigem aprovação; failsafe entre provedores; nome do paciente removido do texto enviado.
- **Preço e cobrança no servidor**: o cliente nunca informa valores.
- **Lógica pura separada** (`*Logic.js`), testável sem banco.
- **Uma instância** por ambiente: presença e WebSocket em memória; jobs no processo ou em worker único.

Perfis do sistema (RBAC)

Grupo (roles.js)	Perfis	Alcance
HEALTH	medico, fisio, enfermeiro, nutricionista, farmaceutico, profissional	Dado clínico dos pacientes vinculados
CARE	HEALTH + cuidador	Agenda, tarefas, evolução e chat (cuidador sem prontuário completo)
ADMIN_STAFF	recepcao, funcionario	Cadastro de pacientes e financeiro; sem dado clínico
STAFF	CARE + ADMIN_STAFF + admin	Equipe: contatos, grafo, histórico
ALL	STAFF + paciente	Todos os perfis
DOCS	medico: receita, atestado, pedido_exames, relatorio; fisio: atestado, relatorio; demais HEALTH: relatorio	Documentos que cada perfil emite

O **root** é um administrador marcado com `users.is_root` (índice único `one_root`): único que cria administradores, redefine a própria senha e restaura a configuração; não pode ser bloqueado.

2 Estrutura do repositório

Onde fica cada coisa

```
talk-m3d/
├─ src/                  servidor
│  ├─ start.js, server.js   pontos de entrada (boot ou app completo)
│  ├─ app.js                Fastify: plugins, hooks, rotas, jobs, listen
│  ├─ boot/                 servidor de instalação (sem banco/chaves)
│  │   ├─ server.js, checks.js, config.js
│  ├─ routes/               23 módulos de rotas (um por domínio)
│  ├─ services/             ai, anomaly, noshow, pdf, sms
│  ├─ *Logic.js             regras puras (billing, caso, consulta, família…)
│  ├─ db.js, crypto.js, audit.js, access.js, roles.js, policy.js
│  ├─ migrate.js, migrateLib.js, seed.js, setup.js, setupLib.js
│  └─ jobs.js, worker.js, presence.js, settings.js, billing.js
├─ db/                   001..013 migrações SQL (ordem lexicográfica)
├─ public/               PWA: index.html, app.js, dash.js, ui2.js, …
├─ tests/                13 arquivos de teste (node --test)
├─ scripts/              check.js, vendor.js, install.sh, update.sh
├─ deploy/               k8s/, oxmgr/, vps-*.sh, gen-secrets.sh, README
├─ docs/                 manuais (md) e exemplos
├─ Dockerfile, docker-compose.yml, Caddyfile
└─ config/usuarios.exemplo.json   modelo de carga inicial
```

Módulos de `src/`

Arquivo	Função
`access.js`	Quem pode ver qual paciente: canSee (CARE vinculado) e canSeeFull (HEALTH vinculado)
`activity.js`	Histórico de atividades (quem, quando, o quê, motivo), sem conteúdo clínico
`app.js`	Composição do servidor Fastify
`assistantLogic.js`	Assistente de ajuda: só indica funções do perfil
`audit.js`	audit(): grava na cadeia com lock; verifyChain(): confere a cadeia
`billing.js`	Banco dos créditos: razão, custo por funcionalidade, bloqueio, renovações
`billingLogic.js`	Regras puras: FEATURES, FEATURE_ROUTES, planos (PLAN_SEED), tokens, acesso
`caseLoad.js`	Carrega dados vinculados ao paciente
`caseLogic.js`	Compila o caso e monta os pedidos à IA
`consultLogic.js`	Minutagem, biblioteca por prioridade, alertas da consulta
`crypto.js`	enc/dec AES-GCM, sha, seal e tokens de convidado HMAC
`db.js`	Pool pg e helper q()
`familyLogic.js`	Prompts, validação e modelo do familiograma e ecomapa
`followup.js`	Receitas e rotinas: baixa por QR, ordem das etapas, notificações
`i18nLogic.js`	Regras puras da tradução de interface
`jobs.js`	Lembretes SMS (15 min), anomalias (10 min), renovações (1 h)
`logic.js`	Estado de receitas e rotinas
`migrateLib.js`	pending() e migrate(): aplica db/*.sql em transação
`policy.js`	Política de senha e cadastro
`presence.js`	Quem está online (WebSocket) e estado escolhido
`reset-root.js`	CLI: nova senha temporária do root
`roles.js`	Perfis e grupos
`settings.js`	Credenciais de integração cifradas em app_settings
`setupLib.js`	createUsers: cria usuários e vínculos (idempotente)
`setupTests.js`	Testa credenciais sem gravar
`testMode.js`	TEST_MODE: contas de teste
`worker.js`	Processo de jobs separado da API
`boot/checks.js`	Verificações pré-instalação (por quê/como resolver)
`boot/config.js`	Lê/grava data/config.env, gera chaves, código de instalação
`boot/server.js`	Servidor de instalação (modo bootstrap)

Módulos de `src/routes/`

Rota (módulo)	Domínio
`admin`	Usuários, aprovação, reset de senha, vínculos, alertas, auditoria
`agenda`	Consultas: criar e mudar status
`app`	Contatos, agenda semanal/atual, notas, resumo e documentos do paciente
`assistente`	Assistente de ajuda
`auth`	Login, logout, /me, senha, status, consentimento
`billing`	Créditos, planos, tokens, embaixadores, gestão
`caso`	Caso clínico e país
`chat`	Canais, mensagens, fixar, traduzir, WebSocket
`clinical`	Estruturação por IA e prontuário (aprovação)
`consulta`	Estado, transcrição, relatório, apoio, biblioteca
`context`	Visão geral, plantão, tarefas, atividade, grafo
`documents`	Receitas, atestados, pedidos: criar, PDF, verificar
`familia`	Familiograma, ecomapa, ordem de protocolos
`finance`	Pacotes, pagamentos, resumo
`followup`	Notificações, rotinas e QR, seguimento
`i18n`	Idioma e tradução de interface
`integracoes`	Chaves de serviços
`paypal`	Pedidos e captura PayPal
`public`	Páginas abertas por QR
`setup`	Assistente de instalação
`signup`	Cadastro público
`tasks`	Exercícios, atribuições, evolução e metas
`video`	Token Agora, convite, convidado, áudio, transcrição

Cliente em `public/`

Arquivo	Linhas	Papel
`index.html`	13	Casca da PWA (CSP: sem scripts inline)
`app.js`	564	Núcleo: estado S, api(), login, render, chat, agenda, helpers de layout
`dash.js`	364	Desktop: post-its, dock, quadros
`ui2.js`	265	Componentes de interface, submenus
`wizard.js`	184	Assistente de instalação
`help.js`	114	Ajuda guiada (?) e demonstração
`assist.js`	51	Chat do assistente
`caso.js`	148	Caso clínico
`consulta.js`	219	Teleconsulta (Agora) e barra da consulta
`familia.js`	86	Familiograma e ecomapa
`creditos.js`	109	Créditos e gestão de créditos
`i18n.js`	94	Cliente de tradução de interface
`i18n-dict.js`	2847	Dicionário base (pt, es, gn, en, zh)
`sw.js`	5	Service worker (cache talk-m3d-vNN)
`app.css`	467	Estilos
`manifest.webmanifest`	1	Manifesto PWA

3 Inicialização, configuração e instalação

Boot em duas fases e variáveis de ambiente

O ponto de entrada (`src/start.js` ou `src/server.js`, usado por PM2/systemd) carrega `data/config.env`, confere as quatro variáveis obrigatórias e decide entre o **servidor de instalação** e o **aplicativo completo**.

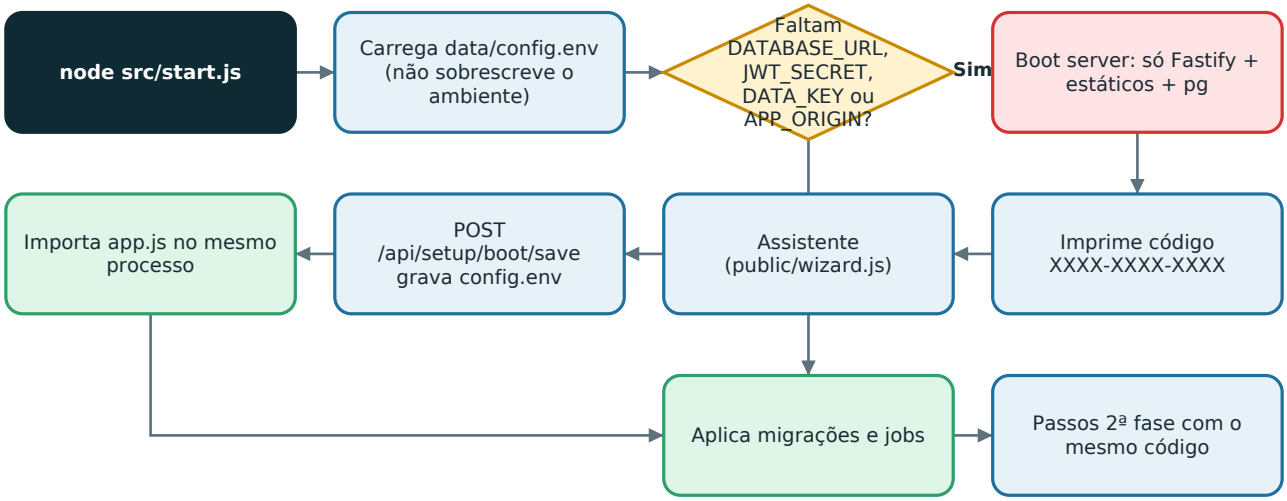


Figura 3.1 · Boot em duas fases. O sistema completo sobe no mesmo processo, sem reiniciar o contêiner.

Detalhes do boot

- `data/config.env` (permissão 0600, escrita atômica): `parseEnv`, `serializeEnv`, `saveConfig`. **Variáveis de ambiente reais têm prioridade** sobre o arquivo.
- `missing()` valida: `DATABASE_URL` com esquema postgres, `JWT_SECRET` ≥ 32, `DATA_KEY` com 32 bytes em base64, `APP_ORIGIN` como `http(s)://host` sem barra.
- `genKeys()` gera `JWT_SECRET` (48 bytes base64url) e `DATA_KEY` (32 bytes base64); `newCode()` gera o código de instalação (12 caracteres sem ambiguidade, em três blocos).
- O código é persistido em `data/setup-code.txt` e reutilizado entre reinícios; comparação em tempo constante e limite de tentativas por IP.
- Banco em loopback (`127.0.0.1`, `localhost`): TLS dispensado automaticamente (`PGSSL=false`, `sslmode` removido da URL).
- O Nginx deve enviar `Host`, `X-Forwarded-Proto $scheme`, `Upgrade` e `Connection` (WebSocket).
- Refazer a instalação: apague `data/config.env` e reinicie, ou use “Restaurar a configuração original” (root).

Diagnóstico preventivo (`src/boot/checks.js`)

Verificação	Nível	Por que importa
Node ≥ 22	obrigatório	Módulos ES, fetch e criptografia nativa
Pasta data/ gravável	obrigatório	Guarda as chaves geradas
Memória ≥ 512 MB	recomendado	Hash de senhas e PDFs
Processo sem root	recomendado	Limita estrago em invasão
Fuso horário (TZ)	recomendado	Lembretes e relatórios na hora local
Proxy HTTPS	recomendado	Certificado e cookies seguros
SDK de vídeo (public/vendor)	recomendado	Sem ele não há teleconsulta
Banco: conexão, versão, TLS, tabela de prova	obrigatório	Nada funciona sem banco
Migrações em dia	obrigatório	Tabelas na versão do código
Chaves, senhas fracas, HTTPS, cookie Secure	mistos	Segurança mínima

Variáveis de ambiente

Variável	Tipo	Descrição
DATABASE_URL	obrig.	Connection string PostgreSQL
JWT_SECRET	obrig.	Segredo do JWT (≥ 32 caracteres) e base das assinaturas de convidado
DATA_KEY	obrig.	Chave AES-256 (32 bytes, base64). Perdê-la torna os dados cifrados irrecuperáveis
APP_ORIGIN	obrig.	Origem pública (https://host); usada em CORS por origem e no WebSocket
PORT / HOST	opc.	Porta (padrão 3000) e interface (padrão 0.0.0.0)
TZ	opc.	Fuso para lembretes e relatórios (padrão America/Sao_Paulo)
DATA_DIR	opc.	Pasta de config.env e setup-code.txt (padrão ./data)
PGSSL	opc.	false desliga TLS do banco (banco local)
COOKIE_SECURE	opc.	false remove o flag Secure do cookie (só para HTTP local)
AUTO_MIGRATE	opc.	false impede migrações ao iniciar
RUN_JOBS	opc.	false: jobs só no worker
SIGNUP_MODE	opc.	off, approval ou open (cadastro público)
BILLING_ENFORCE	opc.	Liga a cobrança por créditos
ALLOW_WEAK_PASSWORDS	opc.	Aceita senhas curtas (só teste/local)
TEST_MODE	opc.	Contas de teste e acesso rápido (só local)
SETUP_CODE	opc.	Fixa o código de instalação
ADMIN_EMAIL / ADMIN_PASSWORD	opc.	Usados por src/seed.js
AGORA_APP_ID / AGORA_APP_CERTIFICATE	integ.	Vídeo
GEMINI_API_KEY / OPENAI_API_KEY / ANTHROPIC_API_KEY	integ.	Provedores de IA
GEMINI_MODEL / OPENAI_MODEL / OPENAI_BASE_URL / ANTHROPIC_MODEL	opc.	Modelos e endpoint
AI_PROVIDERS / STT_PROVIDERS	opc.	Ordem dos provedores (padrão gemini,openai,anthropic / openai,gemini)
AI_TIMEOUT_MS / AI_LONG_TIMEOUT_MS / AI_STT_MODEL	opc.	Tempos limite e modelo de transcrição
PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET / PAYPAL_ENV	integ.	Pagamento
TWILIO_SID / TWILIO_TOKEN / TWILIO_FROM	integ.	SMS
I18N_DAILY_LIMIT	opc.	Limite diário de traduções de interface
ANOM_PATIENTS_10MIN	opc.	Pacientes lidos em 10 min que disparam bloqueio (padrão 30)
PROTOCOL_ORDER	opc.	Hierarquia de protocolos
AMB_PLAN, AMB_*	opc.	Limites do programa de embaixadores
DOMAIN, APP_PORT, HTTP_PORT, HTTPS_PORT	compose	Caddy e portas do host

Integrações podem ser guardadas cifradas no banco (`app_settings`) pelo assistente ou por Integrações; o valor salvo vale imediatamente (`src/settings.js`).

4 Servidor: camadas e ciclo de requisição

src/app.js, plugins, hooks e tratamento de erros

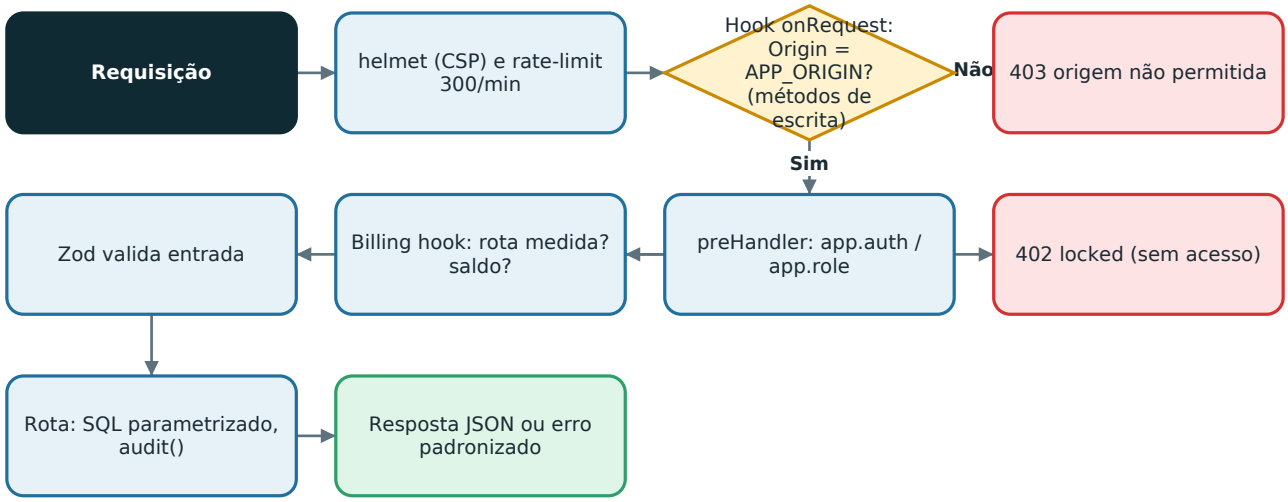


Figura 4.1 · Ciclo de uma requisição à API.

Plugins e configuração (`src/app.js`)

Item	Configuração
Fastify	`trustProxy: true`, `bodyLimit: 1e6`, log com redação de cookie e authorization
helmet	CSP: `default-src self`, `script-src self` (sem inline), estilos self + inline + Google Fonts, imagens self/data/blob
cookie + jwt	Cookie `t` (httpOnly, SameSite=Strict, Secure salvo `COOKIE_SECURE=false`), JWT 8 h
rate-limit	300 requisições por minuto por IP; limites menores no login (8/min) e senha (5/min)
websocket	Rota `/ws`; confere `Origin` e sessão
multipart	Arquivos de até 10 MB, 1 por requisição (áudio de consulta)
static	Serve `public/`; GET fora de `/api` e `/ws` cai em `index.html`
Erros	Zod → 400 com detalhes; 4xx devolve a mensagem; 5xx devolve “erro interno” e registra no log

Decoradores

- `app.auth`: verifica o JWT; confere no banco que o usuário está **ativo** e que o token foi emitido **depois** de `sessions_valid_after` (revogação imediata); com a marca `mc` (troca de senha pendente) só libera `/api/me`, `/api/me/password` e `/api/logout`.
- `app.role(...perfis)`: autentica e exige um dos perfis (403 “sem permissão”).

Registro das rotas e ordem de inicialização

As rotas são importadas dinamicamente na ordem: `auth, agenda, chat, video, finance, tasks, clinical, documents, paypal, app, context, followup, public, signup, setup, integracoes, i18n, caso, assistente, consulta, familia, billing, admin`. Depois: estáticos, not found handler, migrações (se `AUTO_MIGRATE` não for false), `loadSettings()`, `initSetup()`, modo de teste, jobs e `listen`.

WebSocket e presença

- Endpoint `GET /ws` (autenticado, mesma origem). O cliente envia `{"join":"sala"}`; o servidor valida o nome da sala e a permissão (`allowed()`) antes de inscrever o socket.
- Salas: `ch:*` (canais) e salas diretas entre duas pessoas; mensagens novas são difundidas para os sockets da sala.
- `src/presence.js` mantém, em memória, o número de conexões por usuário e o estado escolhido (online, busy, offline); o estado também é gravado em `users.presence`. Por isso o servidor deve ser **uma única instância**.

Jobs (`src/jobs.js`, `src/worker.js`)

Job	Intervalo	O que faz
reminders	15 min	SMS para consultas agendadas/confirmadas ainda não lembradas, a 24 h (48 h se o risco de falta ≥ 0,4)
anomalies	10 min	Acesso em massa (bloqueia conta e cria alerta), acesso fora do horário, sem progresso, meta em risco, piora de dor, baixa adesão
renovacoes	1 h	Renova planos gratuitos e expira testes

Health check

`GET /api/health` responde `{ok, up}`; com `?deep=1` consulta o banco e responde 503 se falhar. Sem limite de taxa (usado por Docker e balanceadores).

5 Segurança, autenticação e auditoria

Defesas em camadas

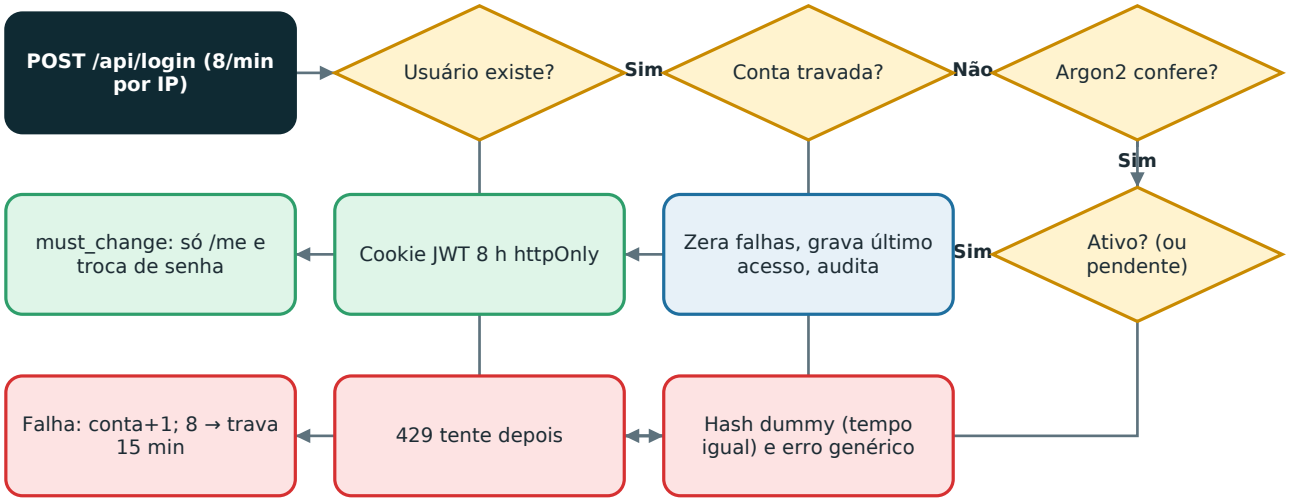


Figura 5.1 · Login. Usuário inexistente e senha errada recebem a mesma resposta; cada falha vai para `credential_events` e `audit_log`.

Controle	Implementação
Senhas	Argon2; mínimo 12 (6 só em servidor local ou com `ALLOW_WEAK_PASSWORDS`); senha temporária gera `must_change`
Sessão	JWT em cookie httpOnly/SameSite=Strict/Secure, 8 h; revogação por `sessions_valid_after`; usuário inativo perde acesso na hora
Bloqueio	`failed_logins` e `locked_until` (8 erros → 15 min); admin destrava
CORS/CSRF	Escritas só com `Origin` igual a `APP_ORIGIN` + SameSite=Strict
CSP	Sem scripts inline; helmet com defaults restritos
RBAC e vínculo	`app.role` por rota + `canSee/canSeeFull` (`care_team`) por paciente; admin e recepção nunca leem dado clínico
Cifragem em repouso	AES-256-GCM: mensagens, transcrições, notas, prontuário, familiograma e integrações. Formato base64(iv[12] ‖ tag[16] ‖ ciphertext)
Convidado de vídeo	Token HMAC-SHA256 (`signGuest/verifyGuest`) com expiração, comparação em tempo constante
QR público	Token aleatório de 144 bits, limite por IP; sem login
Assinatura de documento	SHA-256 do conteúdo + selo HMAC do servidor (`seal`); eletrônica simples, não ICP-Brasil
IA	Texto do usuário tratado só como dado; nome do paciente removido; falha não derruba o atendimento
Anomalias	Leitura em massa bloqueia a conta (exceto admin) e alerta; leituras fora do horário alertam

Auditoria encadeada

Cada ação relevante chama `audit(req, ação, entidade, id, meta)`. A função abre uma transação, obtém um advisory lock (7001), lê o último hash e grava `hash = SHA-256(prev_hash ‖ JSON{usuário, ação, entidade, id, meta, ip, instante})`. Gatilhos do banco impedem `UPDATE`, `DELETE` e `TRUNCATE` em `audit_log`. `GET /api/audit/verify` percorre a cadeia e aponta o primeiro registro adulterado.

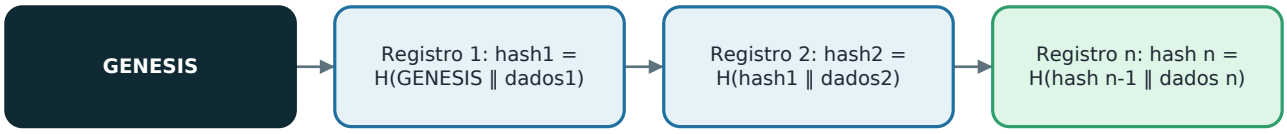


Figura 5.2 · Cadeia de auditoria.

Matriz de acesso às rotas (resumo)

Grupo	Rotas típicas
Pública	login, cadastro e status, páginas QR (`/api/public/*`), `/api/verify/:id`, `/api/setup/*` (código), `/api/video/guest` (token HMAC), health
Sessão	/me, notificações, chat, agenda própria, créditos (`/api/billing/*`), tradução, assistente
HEALTH	vídeo e consulta, caso clínico, estruturação, documentos, familiograma, biblioteca
STAFF	contatos, grafo, histórico, tarefas, agenda semanal
admin + ADMIN_STAFF	usuários (cadastro de pacientes), financeiro
admin	integrações, auditoria, gestão de créditos, alertas, ordem de protocolos
root	criar administradores, restaurar a configuração (`rootOnly`)

6 Domínios funcionais e regras de negócio

Como cada parte funciona por dentro

6.1 Chat

Mensagens ficam em `messages` (`room`, `sender_id`, `body_enc`, `lang`, `pinned`, `urgent`). Só mensagens de canal (`room` começa por `ch:`) podem ser fixadas. A tradução (`GET /api/messages/:id/translate`) decifra e envia à IA com instrução para tratar o texto como conteúdo, nunca como instrução.

6.2 Agenda

Estados: `agendado → confirmado → na_recepcao → em_atendimento → finalizado`, mais `faltou` e `cancelado`. Mudar horário valida conflito e registra o motivo no histórico de atividades. O **risco de falta** (`services/noshow.js`) é uma heurística transparente:

```
score = (faltas + 1) / (consultas passadas + 4)        # suavização bayesiana
se antecedência > 14 dias: score += 0.10
se horário < 8h ou >= 18h:  score += 0.05
risco = min(0.95, score)
risco >= 0.4  =>  lembrete SMS 48 h antes (senão 24 h)
```

6.3 Teleconsulta (`routes/video.js`, `consulta.js`)

- `POST /api/video/token`: só participantes da consulta; gera token RTC Agora (canal `apt-<id>`, uid derivado de SHA-256, 7200 s). Observador recebe SUBSCRIBER.
- `POST /api/video/invite` cria link de especialista (papéis observador, equipe, consultor); `/api/video/guest` troca o token HMAC por token de vídeo.
- `POST /api/consults/:id/audio` recebe áudio (multipart), transcreve por STT com failsafe e grava `transcripts.text_enc`; exige consentimento do paciente (`consult_state` e `PUT /consent`).
- Estruturação, relatório (padrão CARE) e apoio cognitivo usam a IA e a **hierarquia protocolar** configurável (`PROTOCOL_ORDER`), com preferência local do profissional (`clinical_prefs`).

6.4 IA com failsafe (`services/ai.js`)

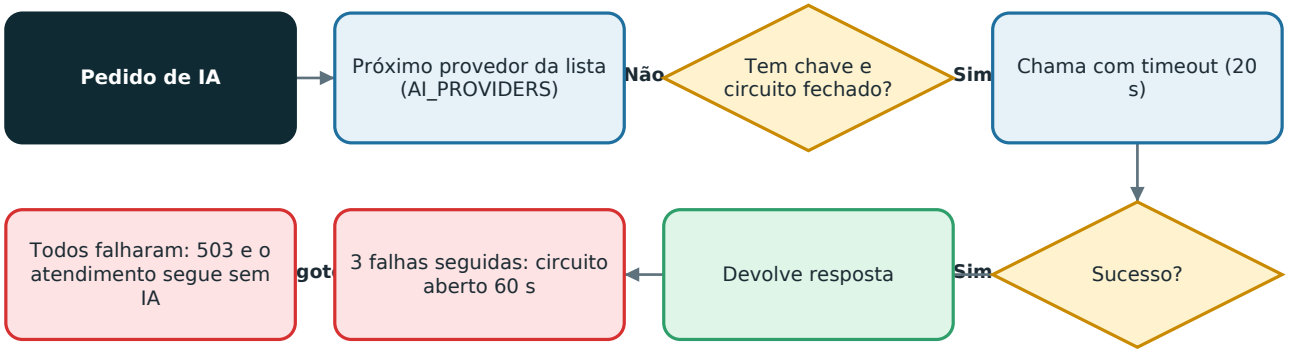


Figura 6.1 · Failsafe de IA. `GET /api/ai/health` mostra provedores e circuitos.

`chat()`, `translate()`, `structure()` e `transcribe()` usam temperatura 0. Provedores: Gemini (`generateContent`), OpenAI (`chat/completions`; STT `whisper-1`) e Anthropic (`messages`). Todas as chamadas são por `fetch` com `AbortSignal.timeout`.

6.5 Prontuário, caso clínico e família

- `POST /api/patients/:id/structure` gera **rascunho** JSON (queixa, sintomas, medicamentos, alergias, EVA, orientações, pendências) em `clinical_entries`; só vira prontuário em `POST /api/clinical/:id/approve`.
- `src/caseLogic.js` compila o que está vinculado ao paciente e monta o pedido à IA (conduta pelo país, alternativas, SOAP, estudo de caso com anonimização). `caseLoad.js` carrega os dados.
- `familyLogic.js`: prompts, validação e saneamento do familiograma e ecomapa; resultado cifrado em `family_maps`.

6.6 Receitas, atestados, pedidos e rotinas

- `POST /api/documents` (perfis de `DOCS`) grava o texto, o SHA-256 e o selo HMAC; define validade e **usos máximos**; gera PDF (`pdfkit`) e QR.
- Página pública do QR: `/api/public/r/:token` (consulta) e `/baixa` (marca uso em `document_uses`, avisa o profissional). Token de 144 bits.
- Rotinas: `routines` → `routine_items` (etapas em ordem) → `routine_events` (execução e dor). `/api/public/p/:token` e `/passo` registram etapas; `logic.js` decide etapa atual e estado.
- `notifications` avisa o profissional (uso de receita, etapa concluída).

6.7 Créditos, planos e tokens

Lógica pura em `billingLogic.js`, banco em `billing.js`, rotas em `routes/billing.js`.

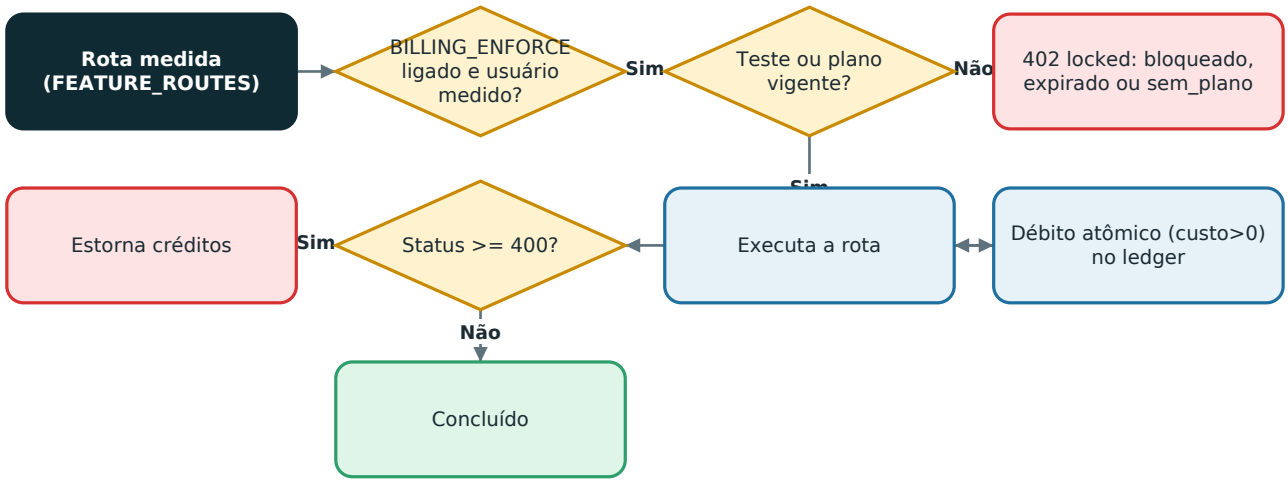


Figura 6.2 · Cobrança por funcionalidade.

Conceito	Regra
Medição	Só quem tem linha em `user_billing`; administradores e usuários sem linha nunca são bloqueados
Custos	`credit_costs` (0 = livre) para: ia_assistente, ia_traducao, ia_transcricao, clinico_ia, consulta_estrutura, consulta_relatorio, consulta_apoio, caso_ia, familia_ia
Razão	`credit_ledger` com tipo (plano, uso, estorno, token, indicação, concessão, ajuste, distribuição); saldo nunca negativo (CHECK + atualização condicional)
Planos	Básico (grátis), Essencial R$ 59/mês ou R$ 19/semana, Profissional R$ 149/R$ 45, Clínica R$ 399/R$ 119; editáveis; semanal ≈ 30% do mensal
Tokens	`signup_tokens` (TALK padrão: 7 dias, plano Básico) e `token_uses`; indicação premia o dono com `reward_owner`
Embaixador	Cria grupos de tokens `AMB-XXXXXX` dentro de limites (`AMB_*`)
Pagamento	PayPal: captura confere moeda BRL e valor; ou pedido `billing_orders` pendente confirmado pelo admin

Conceito	Regra
Rotas abertas quando bloqueado	/api/me, /api/logout, /api/billing/*, login, signup, setup, health, i18n dict/langs

6.8 Financeiro, notificações e tradução de interface

- Financeiro: `packages` (pacotes de sessões), `payments` (entrada/saída em centavos) e GET `/api/finance/summary` calculado no servidor.
- `i18n`: dicionário base em `public/i18n-dict.js`; textos novos são traduzidos por IA e guardados em `i18n_cache`; `i18n_usage` aplica o limite diário. Dados de usuários nunca entram nessa tradução.

7 Referência da API

158 rotas por módulo, com o tipo de acesso

Todas as rotas usam o prefixo `/api` (exceto `/ws`). Entrada validada por Zod; saída JSON. Acesso “perfil: admin, ADMIN_STAFF” significa administrador, recepção e funcionário.

`admin` · Usuários, vínculos, alertas e auditoria

Método	Caminho	Acesso
GET	/api/users	perfil: admin, ADMIN_STAFF
POST	/api/users	perfil: admin, ADMIN_STAFF
PATCH	/api/users/:id/approve	perfil: admin, ADMIN_STAFF
DELETE	/api/users/:id/pending	perfil: admin, ADMIN_STAFF
PATCH	/api/users/:id/active	perfil: admin
POST	/api/users/:id/reset-password	perfil: admin
POST	/api/users/:id/unlock	perfil: admin
POST	/api/users/:id/revoke-sessions	perfil: admin
GET	/api/users/:id/credential-events	perfil: admin
PATCH	/api/users/:id/registro	perfil: admin
POST	/api/care-team	perfil: admin, ADMIN_STAFF
GET	/api/alerts	perfil: admin
PATCH	/api/alerts/:id/ack	perfil: admin
GET	/api/audit	perfil: admin
GET	/api/ai/health	perfil: admin
GET	/api/audit/verify	perfil: admin

`agenda` · Consultas

Método	Caminho	Acesso
GET	/api/agenda	sessão
POST	/api/agenda	perfil: STAFF
PATCH	/api/agenda/:id/status	perfil: STAFF

`app` · Contatos, agenda semanal, notas, resumo

Método	Caminho	Acesso
GET	/api/contacts	sessão
GET	/api/agenda/week	sessão
GET	/api/agenda/current	sessão
PATCH	/api/agenda/:id	perfil: STAFF
POST	/api/agenda/notes	perfil: STAFF
PATCH	/api/agenda/notes/:id	perfil: STAFF
DELETE	/api/agenda/notes/:id	perfil: STAFF
GET	/api/patients/:id/summary	sessão
PUT	/api/patients/:id/summary	perfil: HEALTH
GET	/api/patients/:id/documents	sessão

`assistente` · Assistente de ajuda

Método	Caminho	Acesso
GET	/api/assistant/info	sessão
POST	/api/assistant	sessão

auth · Autenticação e conta

Método	Caminho	Acesso
POST	/api/login	pública (8/min)
POST	/api/logout	sessão
GET	/api/me	sessão
POST	/api/me/password	sessão
PUT	/api/me/status	sessão
PATCH	/api/me/consent	sessão

billing · Créditos, planos, tokens e gestão

Método	Caminho	Acesso
GET	/api/billing/me	sessão
GET	/api/billing/plans	sessão
GET	/api/billing/ledger	sessão
POST	/api/billing/checkout	sessão
POST	/api/billing/orders/:id/capture	sessão
POST	/api/billing/redeem	sessão
GET	/api/billing/ambassador	sessão
POST	/api/billing/ambassador/groups	sessão (embaixador)
PATCH	/api/billing/ambassador/tokens/:code	sessão
GET	/api/admin/billing/overview	perfil: admin
PUT	/api/admin/billing/settings	perfil: admin
GET	/api/admin/billing/plans	perfil: admin
PUT	/api/admin/billing/plans/:id	perfil: admin
GET	/api/admin/billing/costs	perfil: admin
PUT	/api/admin/billing/costs	perfil: admin
GET	/api/admin/billing/users	perfil: admin
POST	/api/admin/billing/users/:id	perfil: admin
POST	/api/admin/billing/distribute	perfil: admin
GET	/api/admin/billing/ledger	perfil: admin
GET	/api/admin/billing/tokens	perfil: admin
POST	/api/admin/billing/tokens	perfil: admin
PATCH	/api/admin/billing/tokens/:code	perfil: admin
GET	/api/admin/billing/orders	perfil: admin
POST	/api/admin/billing/orders/:id/:acao	perfil: admin

caso · Caso clínico

Método	Caminho	Acesso
GET	/api/case/countries	sessão
PUT	/api/me/country	sessão
GET	/api/patients/:id/case	perfil: HEALTH
POST	/api/patients/:id/case/ai	perfil: HEALTH (+ créditos)

chat · Chat e canais

Método	Caminho	Acesso
GET	/api/channels	perfil: STAFF
POST	/api/channels	perfil: admin
GET	/api/messages	sessão
POST	/api/messages	sessão
PATCH	/api/messages/:id/pin	perfil: STAFF
GET	/api/messages/:id/translate	sessão
GET	/ws	sessão

clinical · Prontuário

Método	Caminho	Acesso
POST	/api/patients/:id/structure	perfil: HEALTH
POST	/api/clinical/:id/approve	perfil: HEALTH
GET	/api/patients/:id/clinical	sessão

consulta · Consulta, relatório e biblioteca

Método	Caminho	Acesso
GET	/api/consults/:id/state	sessão
PUT	/api/consults/:id/transcription	perfil: HEALTH
PUT	/api/consults/:id/consent	sessão
GET	/api/consults/:id/live	sessão (participante)
GET	/api/consults/:id/minutes	sessão (participante)
POST	/api/consults/:id/structure	perfil: HEALTH (+ créditos)
POST	/api/consults/:id/report	perfil: HEALTH (+ créditos)
GET	/api/consults/:id/report	perfil: HEALTH (+ créditos)
PUT	/api/consults/:id/report	perfil: HEALTH (+ créditos)
POST	/api/consults/:id/support	perfil: HEALTH (+ créditos)
GET	/api/library	perfil: HEALTH
POST	/api/library	perfil: HEALTH
DELETE	/api/library/:id	sessão
GET	/api/me/clinical-prefs	perfil: HEALTH
PUT	/api/me/clinical-prefs	perfil: HEALTH

context · Contexto, tarefas e grafo

Método	Caminho	Acesso
GET	/api/overview	perfil: STAFF
GET	/api/shift	perfil: STAFF
GET	/api/tasks	perfil: STAFF
POST	/api/tasks	perfil: STAFF
PATCH	/api/tasks/:id	perfil: STAFF
DELETE	/api/tasks/:id	perfil: STAFF
GET	/api/activity	perfil: STAFF
PATCH	/api/me/ask-reason	sessão
PATCH	/api/users/:id/ask-reason	perfil: admin
GET	/api/graph	perfil: STAFF

documents · Documentos com QR

Método	Caminho	Acesso
POST	/api/documents	perfil: perfis de DOCS
GET	/api/documents/:id/pdf	sessão
GET	/api/verify/:id	pública (verificação)

familia · Familiograma e protocolos

Método	Caminho	Acesso
GET	/api/patients/:id/family-map	perfil: HEALTH
POST	/api/patients/:id/family-map/generate	perfil: HEALTH (+ créditos)
PUT	/api/patients/:id/family-map	perfil: HEALTH
GET	/api/protocol-order	sessão
GET	/api/admin/protocol-order	perfil: admin
PUT	/api/admin/protocol-order	perfil: admin

finance · Financeiro

Método	Caminho	Acesso
POST	/api/packages	perfil: admin, ADMIN_STAFF
GET	/api/packages	perfil: admin, ADMIN_STAFF
POST	/api/payments	perfil: admin, ADMIN_STAFF
GET	/api/finance/summary	perfil: admin, ADMIN_STAFF

followup · Notificações e rotinas

Método	Caminho	Acesso
GET	/api/notifications	sessão
GET	/api/notifications/count	sessão
PATCH	/api/notifications/:id/read	sessão
POST	/api/notifications/read-all	sessão
POST	/api/routines	perfil: HEALTH
GET	/api/routines	sessão
POST	/api/routines/:id/passo	sessão
GET	/api/routines/:id/qr.svg	sessão
GET	/api/documents/:id/qr.svg	sessão
GET	/api/patients/:id/followup	sessão

i18n · Idioma e tradução

Método	Caminho	Acesso
PATCH	/api/me/lang	sessão
POST	/api/i18n/translate	sessão

integracoes · Integrações

Método	Caminho	Acesso
GET	/api/admin/settings	perfil: admin
PUT	/api/admin/settings	perfil: admin
POST	/api/admin/settings/test	perfil: admin

paypal · PayPal

Método	Caminho	Acesso
POST	/api/paypal/orders	perfil: paciente, admin, ADMIN_STAFF
POST	/api/paypal/orders/:id/capture	perfil: paciente, admin, ADMIN_STAFF

public · Páginas por QR

Método	Caminho	Acesso
GET	/api/public/r/:token	pública (token do QR)
POST	/api/public/r/:token/baixa	pública (token do QR)
GET	/api/public/p/:token	pública (token do QR)
POST	/api/public/p/:token/passo	pública (token do QR)

`setup` · Instalação

Método	Caminho	Acesso
POST	/api/admin/reset-install	root (senha)
POST	/api/admin/cancel-reset	root (senha)
GET	/api/setup/status	código de instalação
POST	/api/setup/unlock	código de instalação
POST	/api/setup/migrate	código de instalação
GET	/api/setup/check	código de instalação
POST	/api/setup/test	código de instalação
POST	/api/setup/save	código de instalação
POST	/api/setup/finish	código de instalação

`signup` · Cadastro público

Método	Caminho	Acesso
GET	/api/signup/status	pública (modo de cadastro)
POST	/api/signup	pública (modo de cadastro)

`tasks` · Exercícios, evolução e metas

Método	Caminho	Acesso
GET	/api/exercises	sessão
POST	/api/exercises	perfil: HEALTH
POST	/api/assignments	perfil: HEALTH
GET	/api/me/tasks	perfil: paciente
POST	/api/assignments/:id/log	perfil: paciente
POST	/api/patients/:id/evolution	perfil: CARE
POST	/api/patients/:id/goal	perfil: HEALTH
GET	/api/patients/:id/dashboard	sessão

`video` · Videochamada

Método	Caminho	Acesso
POST	/api/video/token	sessão
POST	/api/video/invite	perfil: HEALTH
POST	/api/video/guest	token HMAC do convite
POST	/api/consults/:id/audio	sessão
GET	/api/consults/:id/transcript	perfil: HEALTH

Também: `GET /ws` (WebSocket), `GET /api/health`, `GET /api/test-users` (só com `TEST_MODE`) e, no servidor de instalação, `/api/setup/boot/*`.

8 Banco de dados

PostgreSQL: modelo, tabelas, índices e migrações

O esquema vive em `db/NNN_*.sql` (13 migrações, 40 tabelas). `migrateLib.js` aplica, em ordem e cada uma em transação, as que faltam, registrando em `_migrations(name, at)`. Convenções: chaves `uuid` (`gen_random_uuid()`), datas `timestamptz`, dinheiro em centavos (`*_cents`) ou BRL inteiro nos planos, dados sensíveis em colunas `*_enc` (AES-GCM).

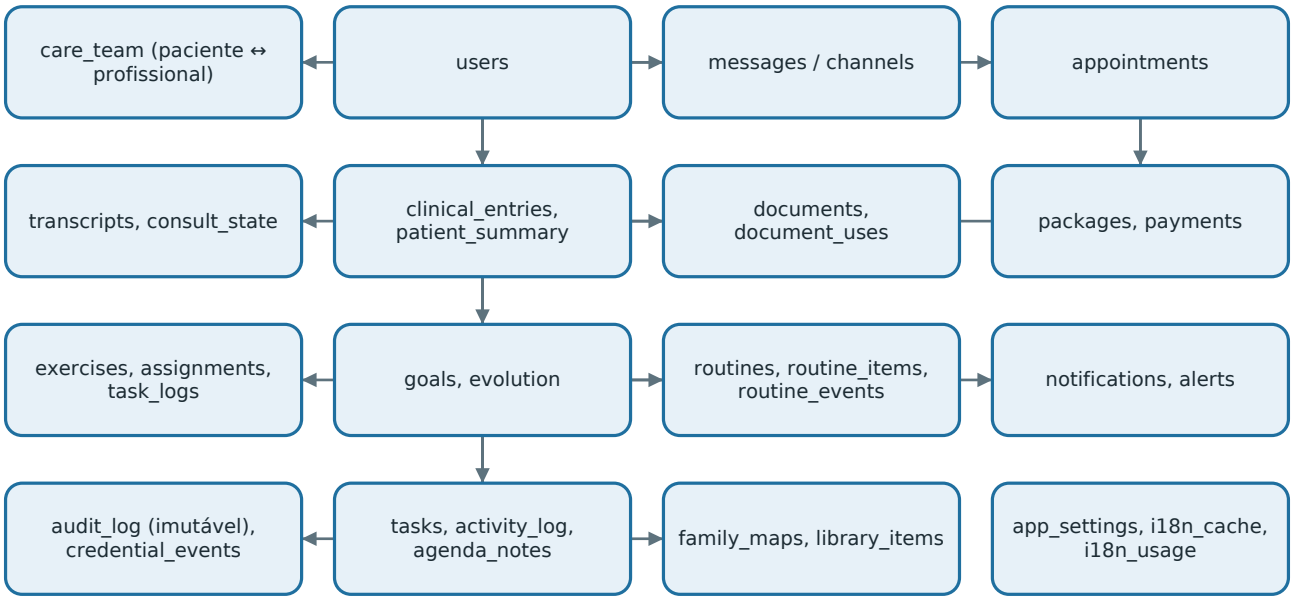


Figura 8.1 · Grupos de tabelas ligados a `users` (a maioria das tabelas tem `patient_id`/`user_id` com chave estrangeira).

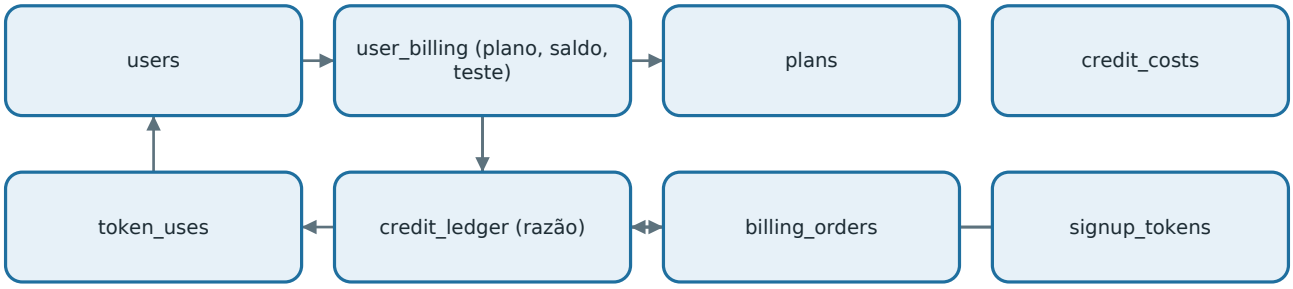


Figura 8.2 · Módulo de créditos (migração 012).

Mig.	Conteúdo
001	Núcleo: users, care_team, channels, messages, packages, appointments, payments, transcripts, clinical_entries, exercises, assignments, task_logs, goals, evolution, documents, alerts, audit_log + gatilhos de imutabilidade
002	agenda_notes e histórico de agenda
003	patient_summary, activity_log, tasks
004	Papéis ampliados, cargo, is_root (índice único one_root), must_change
005	document_uses, routines, routine_items, routine_events, notifications; registro profissional e assinatura
006	app_settings (integrações cifradas)
007	Cadastro público: username (único, minúsculo), pending, failed_logins, locked_until
008	i18n_cache e i18n_usage
009	presence e country em users
010	consult_state, library_items, clinical_prefs

Mig.	Conteúdo
011	family_maps
012	plans, credit_costs, user_billing, credit_ledger, signup_tokens, token_uses, billing_orders (+ planos, custos e token TALK)
013	credential_events; last_login_at/ip, pw_changed_at, sessions_valid_after

Índices, restrições e gatilhos

- `messages(room, created_at desc)`, `appointments(starts_at)`, `agenda_notes(owner_id, starts_at)`, `activity_log(at desc)` e `(patient_id)`, `document_uses(document_id, at desc)`, `notifications(user_id, created_at desc)`, `credit_ledger(user_id, id desc)`, `signup_tokens(owner_id)`, `credential_events(user_id, id desc)`, `library_items(owner_id)`.
- Únicos: `users.email`, `one_root` (`is_root` verdadeiro), `users_username_key` (`lower(username)` quando preenchido), `channels.name`.
- CHECKs: perfis de `users.role`, eva e metas entre 0 e 10, `total_sessions > 0`, `amount_cents > 0`, saldo de créditos não negativo.
- Gatilhos: `audit_row` (antes de UPDATE/DELETE) e `audit_trunc` (antes de TRUNCATE) chamam `audit_no_mutation()`, que levanta exceção.

Colunas cifradas (AES-256-GCM)

`messages.body_enc`, `appointments.note_enc`, `transcripts.text_enc`, `transcripts.translated_enc`, `clinical_entries.content_enc`, `agenda_notes.body_enc`, `patient_summary.content_enc`, `app_settings.value_enc`, `consult_state.structured_enc`, `consult_state.report_enc`, `family_maps.data_enc`. Outras colunas sensíveis (como as chaves em `app_settings`) também são cifradas pelo código.

Dicionário de dados

Tipos e restrições como declarados nas migrações; “[+nnn]” indica coluna adicionada pela migração nnn.

users (migração 001)	
id	uuid primary key default gen_random_uuid()
role	text not null check(role in('admin','medico','fisio','recepcao','paciente'))
name	text not null
email	text unique not null
phone	text
pw_hash	text not null
active	boolean not null default true
lang	text not null default 'pt'
ai_consent	boolean not null default false
created_at	timestamptz not null default now()
ask_reason	boolean not null default true [+003]
cargo	text [+004]
is_root	boolean not null default false [+004]
must_change	boolean not null default false [+004]
reg_conselho	text [+005]
reg_numero	text [+005]
reg_uf	text [+005]
sign_img	text [+005]
icp_ref	text [+005]
username	text [+007]
pending	boolean not null default false [+007]
failed_logins	int not null default 0 [+007]
locked_until	timestamptz [+007]
country	text check (country is null or country ~ '^[A-Z]{2}$') [+009]
clinical_prefs	jsonb not null default '{}'::jsonb [+010]
last_login_at	timestamptz [+013]
last_login_ip	text [+013]
pw_changed_at	timestamptz [+013]
sessions_valid_after	timestamptz [+013]

care_team (migração 001)	
patient_id	uuid references users(id)
professional_id	uuid references users(id)
restrição	primary key(patient_id,professional_id)

channels (migração 001)	
id	uuid primary key default gen_random_uuid()
name	text unique not null

messages (migração 001)	
id	uuid primary key default gen_random_uuid()
room	text not null
sender_id	uuid not null references users(id)
body_enc	text not null
lang	text
pinned	boolean not null default false
urgent	boolean not null default false
created_at	timestamptz not null default now()

packages (migração 001)	
id	uuid primary key default gen_random_uuid()
patient_id	uuid not null references users(id)
name	text not null
total_sessions	int not null check(total_sessions>0)
used_sessions	int not null default 0
price_cents	int not null
created_at	timestamptz not null default now()
restrição	check(used_sessions<=total_sessions)

appointments (migração 001)	
id	uuid primary key default gen_random_uuid()
patient_id	uuid not null references users(id)
professional_id	uuid not null references users(id)
starts_at	timestamptz not null
ends_at	timestamptz not null
status	text not null default 'agendado' check(status in('agendado','confirmado','na_recepcao', 'em_atendimento','finalizado','faltou','cancelado'))
package_id	uuid references packages(id)
noshow_risk	numeric(3,2)
reminded_at	timestamptz
created_at	timestamptz not null default now()
note_enc	text [+002]

payments (migração 001)	
id	uuid primary key default gen_random_uuid()
kind	text not null check(kind in('entrada','saida'))
amount_cents	int not null check(amount_cents>0)
category	text
method	text
note	text
patient_id	uuid references users(id)
package_id	uuid references packages(id)
paid_at	timestamptz not null default now()

transcripts (migração 001)	
id	uuid primary key default gen_random_uuid()
appointment_id	uuid not null references appointments(id)
speaker_id	uuid not null references users(id)
lang	text
text_enc	text not null
translated_enc	text
to_lang	text
created_at	timestamptz not null default now()

clinical_entries (migração 001)	
id	uuid primary key default gen_random_uuid()
patient_id	uuid not null references users(id)
author_id	uuid not null references users(id)
kind	text not null
source	text not null
content_enc	text not null
approved	boolean not null default false
approved_by	uuid references users(id)
created_at	timestamptz not null default now()

exercises (migração 001)	
id	uuid primary key default gen_random_uuid()
title	text not null
description	text
steps	jsonb not null default '[]'
video_url	text
created_by	uuid references users(id)

assignments (migração 001)	
id	uuid primary key default gen_random_uuid()
patient_id	uuid not null references users(id)
exercise_id	uuid not null references exercises(id)
professional_id	uuid not null references users(id)
freq_per_week	int not null
sets	int not null
reps	int not null
start_date	date not null
end_date	date not null
goal	text
active	boolean not null default true

task_logs (migração 001)	
id	uuid primary key default gen_random_uuid()
assignment_id	uuid not null references assignments(id)
done_at	timestamptz not null default now()
pain	int check(pain between 0 and 10)
note	text

goals (migração 001)	
id	uuid primary key default gen_random_uuid()
patient_id	uuid not null references users(id)
description	text not null
target	int not null check(target between 0 and 10)
due	date not null
active	boolean not null default true

evolution (migração 001)	
id	uuid primary key default gen_random_uuid()
patient_id	uuid not null references users(id)
at	timestamptz not null default now()
eva	int not null check(eva between 0 and 10)
note	text
source	text not null default 'profissional'

documents (migração 001)	
id	uuid primary key default gen_random_uuid()
patient_id	uuid not null references users(id)
author_id	uuid not null references users(id)
kind	text not null
texto	text not null
hash	text not null
seal	text not null
created_at	timestamptz not null
token	text unique [+005]
max_uses	int not null default 1 check(max_uses between 1 and 12) [+005]
uses	int not null default 0 [+005]
valid_until	timestamptz [+005]

alerts (migração 001)	
id	uuid primary key default gen_random_uuid()
kind	text not null
severity	text not null
patient_id	uuid references users(id)
user_id	uuid references users(id)
detail	json
ack	boolean not null default false
created_at	timestamptz not null default now()

audit_log (migração 001)	
id	bigserial primary key
at	timestamptz not null
user_id	uuid
action	text not null
entity	text not null
entity_id	text
ip	text
meta	json
prev_hash	text not null
hash	text not null

agenda_notes (migração 002)	
id	uuid primary key default gen_random_uuid()
owner_id	uuid not null references users(id)
title	text not null default 'Nota'
body_enc	text not null default ''
starts_at	timestamptz not null
created_at	timestamptz not null default now()

patient_summary (migração 002)	
patient_id	uuid primary key references users(id)
content_enc	text not null
updated_by	uuid references users(id)
updated_at	timestamptz not null default now()

activity_log (migração 003)	
id	bigserial primary key
at	timestamptz not null default now()
user_id	uuid references users(id)
entity	text not null
entity_id	text
action	text not null
patient_id	uuid references users(id)
detail	json
reason	text

tasks (migração 003)	
id	uuid primary key default gen_random_uuid()
title	text not null
status	text not null default 'todo' check(status in('todo','doing','done'))
assignee_id	uuid references users(id)
patient_id	uuid references users(id)
channel	text
due	date
created_by	uuid references users(id)
created_at	timestamptz not null default now()

document_uses (migração 005)	
id	bigserial primary key
document_id	uuid not null references documents(id)
at	timestamptz not null default now()
action	text not null check(action in('utilizada','nao_utilizada'))
by_name	text not null
by_place	text
ip	text

routines (migração 005)	
id	uuid primary key default gen_random_uuid()
patient_id	uuid not null references users(id)
professional_id	uuid not null references users(id)
title	text not null
token	text unique not null
status	text not null default 'ativa' check(status in('ativa','concluida','cancelada'))
created_at	timestamptz not null default now()
finished_at	timestamptz

routine_items (migração 005)	
id	uuid primary key default gen_random_uuid()
routine_id	uuid not null references routines(id) on delete cascade
position	int not null
title	text not null
exercise_id	uuid references exercises(id)
sessions	int not null check(sessions between 1 and 60)
done	int not null default 0
restrição	unique(routine_id,position)

routine_events (migração 005)	
id	bigserial primary key
item_id	uuid not null references routine_items(id) on delete cascade
at	timestamptz not null default now()
action	text not null check(action in('concluida','nao_realizada'))
by_name	text not null
note	text

notifications (migração 005)	
id	uuid primary key default gen_random_uuid()
user_id	uuid not null references users(id)
kind	text not null
title	text not null
body	text
patient_id	uuid references users(id)
ref	text
created_at	timestamptz not null default now()
read_at	timestamptz

app_settings (migração 006)	
key	text primary key
value_enc	text not null
updated_at	timestamptz not null default now()

i18n_cache (migração 008)	
lang	text not null
src_hash	text not null
src	text not null
ctx	text
out	text not null
created_at	timestamptz not null default now()
restrição	primary key(lang, src_hash)

i18n_usage (migração 008)	
day	date not null default current_date
user_id	uuid not null references users(id)
n	int not null default 0
restrição	primary key(day, user_id)

consult_state (migração 010)	
appointment_id	uuid primary key references appointments(id) on delete cascade
transcribing	boolean not null default false
patient_consent	boolean
started_at	timestamptz
structured_enc	text
structured_at	timestamptz
report_enc	text
report_at	timestamptz

library_items (migração 010)	
id	uuid primary key default gen_random_uuid()
owner_id	uuid not null references users(id)
tier	text not null check (tier in ('ministerio','sociedade','oms','personal'))
shared	boolean not null default false
country	text check (country is null or country ~ '^[A-Z]{2}$')
title	text not null
body	text not null
created_at	timestamptz not null default now()

family_maps (migração 011)	
patient_id	uuid not null references users(id) on delete cascade
kind	text not null check (kind in ('genograma','ecomapa'))
data_enc	text not null
source	text not null default 'ia' check (source in ('ia','manual'))
lib_used	int not null default 0
updated_by	uuid references users(id)
updated_at	timestamptz not null default now()
restrição	primary key (patient_id, kind)

plans (migração 012)	
id	text primary key
name	text not null
tier	int not null default 1
description	text not null default ''
price_week	numeric(10,2) not null default 0
price_month	numeric(10,2) not null default 0
credits_week	int not null default 0
credits_month	int not null default 0
features	jsonb not null default '{}'
active	boolean not null default true
sort	int not null default 0

credit_costs (migração 012)	
feature	text primary key
label	text not null
cost	int not null default 0 check (cost >= 0)

user_billing (migração 012)	
user_id	uuid primary key references users(id) on delete cascade
plan_id	text references plans(id)
cycle	text check (cycle in ('week','month'))
balance	int not null default 0 check (balance >= 0)
trial_ends	timestamptz
plan_ends	timestamptz
auto_renew	boolean not null default true
blocked	boolean not null default false
blocked_reason	text
ambassador	boolean not null default false
referred_by	uuid references users(id)
token_used	text
updated_at	timestamptz not null default now()

credit_ledger (migração 012)	
id	bigserial primary key
user_id	uuid not null references users(id) on delete cascade
delta	int not null
balance_after	int not null
kind	text not null
feature	text
note	text
created_by	uuid references users(id)
created_at	timestamptz not null default now()

signup_tokens (migração 012)	
code	text primary key
kind	text not null default 'promo' check (kind in ('default','promo','embaixador'))
label	text not null default ''
trial_days	int not null default 0 check (trial_days >= 0)
plan_id	text references plans(id)
credits	int not null default 0 check (credits >= 0)
reward_owner	int not null default 0 check (reward_owner >= 0)
owner_id	uuid references users(id) on delete set null
group_id	text
max_uses	int
uses	int not null default 0
expires_at	timestamptz
active	boolean not null default true
created_by	uuid references users(id)
created_at	timestamptz not null default now()

billing_orders (migração 012)	
id	bigserial primary key
user_id	uuid not null references users(id) on delete cascade
plan_id	text not null references plans(id)
cycle	text not null check (cycle in ('week','month'))
amount	numeric(10,2) not null
status	text not null default 'pendente' check (status in ('pendente','pago','cancelado'))
method	text
note	text
created_at	timestamptz not null default now()
paid_at	timestamptz

token_uses (migração 012)	
code	text not null references signup_tokens(code) on delete cascade
user_id	uuid not null references users(id) on delete cascade
at	timestamptz not null default now()
restrição	primary key (code, user_id)

credential_events (migração 013)	
id	bigserial primary key
user_id	uuid not null references users(id) on delete cascade
kind	text not null check (kind in ('login','login_falha','bloqueio','desbloqueio','senha_trocada','senha_redefinida','sessao_revogada'))
ip	text
by_user	uuid references users(id)
at	timestamptz not null default now()

9 Cliente (PWA)

Estrutura, estado e padrões do front-end

O cliente não usa framework nem etapa de build: arquivos `public/*.js` carregados por `index.html` (sem script inline, por causa da CSP). O estado global fica em `S`; chamadas ao servidor passam por `api(caminho, {m, b})`, que envia o cookie e trata 401 (volta ao login) e 402 (abre Créditos).

Conceito	Como é no código
Estado	Objeto `S` (usuário `S.me`, visão `S.view`, contatos, agenda…); `render()` redesenha por template string
Ações	Objeto `A` com métodos (`A.view(p)` troca de tela); delegação de eventos por `data-a`/`data-i`
Layout do usuário	`lay()`/`saveLay()` em `localStorage`: blocos do menu, `autohide`, `dhide` (itens do desktop ocultos), posição de post-its (`PIT`)
Desktop	`dash.js`: post-its lado a lado (empacotamento em linhas), dock, quadros arrastáveis, `HOME_APPS`
Dock e submenu	Um ícone por função do perfil; clique abre/minimiza; 3 cliques fecham; quadros assumem a cor do ícone
Tradução da interface	`i18n.js` aplica `i18n-dict.js`; faltas vão ao servidor e voltam em cache
Anotações	Só em `localStorage`, nunca no servidor
Teleconsulta	`consulta.js` usa o SDK em `public/vendor/agora.js` (copiado por `scripts/vendor.js` no `postinstall`)
Instalação	`manifest.webmanifest` + ícones 192/512; `sw.js` com cache `talk-m3d-vNN`

> **Atenção:** Ao alterar qualquer arquivo de `public/`, **aumente o número do cache** em `sw.js` (`talk-m3d-vNN`); senão os navegadores continuam com a versão antiga. No usuário final, recarregue com Ctrl+F5 ou use aba anônima.

Boas práticas do front-end

- Nenhum `onclick` inline nem `eval`: a CSP bloqueia.
- Todo texto vindo de usuário passa por escape antes de entrar em template string.
- Chamadas de IA mostram o aviso “rascunho”; resultados nunca são gravados sem ação do usuário.
- Sem dependências de CDN para scripts; apenas fontes do Google (CSP).

10 Implantação e operação

Docker, PM2 + Nginx, Kubernetes, CI/CD e rotinas

10.1 Docker Compose

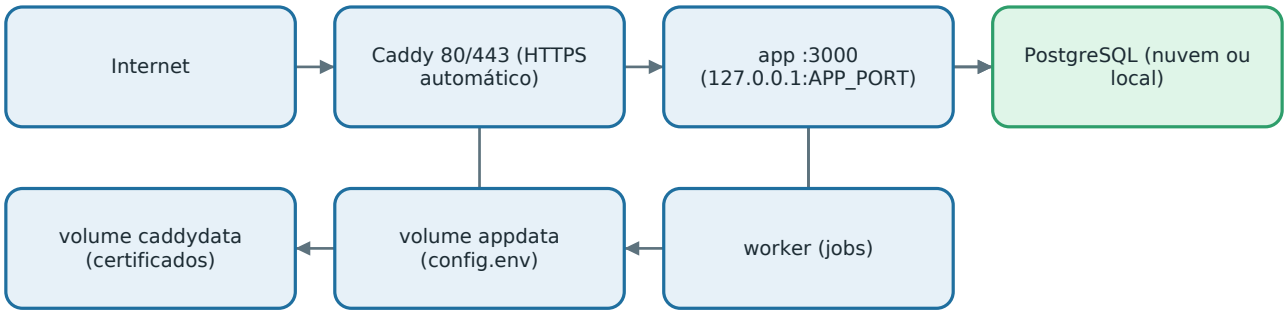


Figura 10.1 · Topologia do `docker-compose.yml`.

- Serviços: app (somente leitura, `cap_drop: ALL`, `no-new-privileges`, limites de memória/CPU/pids, healthcheck), `worker` (uma instância) e caddy.
- Subir: `docker compose up -d --build`. Com HTTPS: `DOMAIN=seu.dominio docker compose up -d`. Logs: `docker compose logs app`.
- Volumes: appdata (config.env e setup-code) e caddydata.

10.2 VPS sem Docker: PM2 + Nginx (ambiente atual)

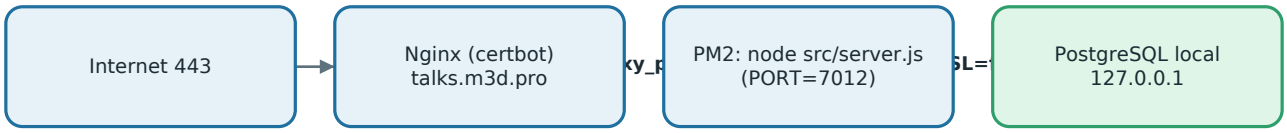


Figura 10.2 · Topologia no VPS. Cada aplicativo do servidor usa uma porta própria; o Nginx de cada subdomínio aponta para ela.

```
# Nginx (location /)
proxy_pass http://127.0.0.1:7012;
proxy_http_version 1.1;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";

# PM2
cd /var/www/talk && PORT=7012 pm2 start src/server.js --name talk && pm2 save
pm2 logs talk --lines 50 | grep INSTALAÇÃO     # código do assistente
```

10.3 Scripts do repositório

Script	Função
`scripts/install.sh`	Gera .env e sobe o Compose
`scripts/update.sh`	Atualiza e reconstrói
`scripts/check.js`	Sintaxe de todos os .js (`npm run check`)
`scripts/vendor.js`	Copia o SDK Agora para public/vendor (postinstall)
`deploy/gen-secrets.sh`	Gera segredos (Kubernetes)
`deploy/vps-setup-deploy.sh`	Configura o deploy por SSH com comando forçado (talk-deploy)
`deploy/vps-fix-proxy.sh`	Corrige o Nginx (X-Forwarded-Proto, WebSocket), com backup fora de sites-enabled
`deploy/vps-criar-banco.sh`	Cria banco e usuário PostgreSQL locais e imprime a connection string
`src/reset-root.js`	Nova senha temporária do root
`node src/setup.js config/usuarios.json`	Carga inicial sem tela

10.4 Kubernetes e systemd

- `deploy/k8s`: `00-base` (base do namespace e volumes), `10-secret.example` (Secret gerado por `gen-secrets.sh`), `20-web` (Deployment da API, 1 réplica, Recreate), `30-worker`, `40-ingress` (entrada e regras de rede) e `kustomization.yaml`.
- `deploy/oxmgr`: `oxfile.toml`, `talk-m3d.service` (systemd com `MemoryMax`, `CPUQuota`, usuário próprio) e `install.sh`.
- Escalar a API em várias réplicas **não** é suportado (presença e WebSocket em memória).

10.5 Deploy automático (GitHub Actions → VPS)

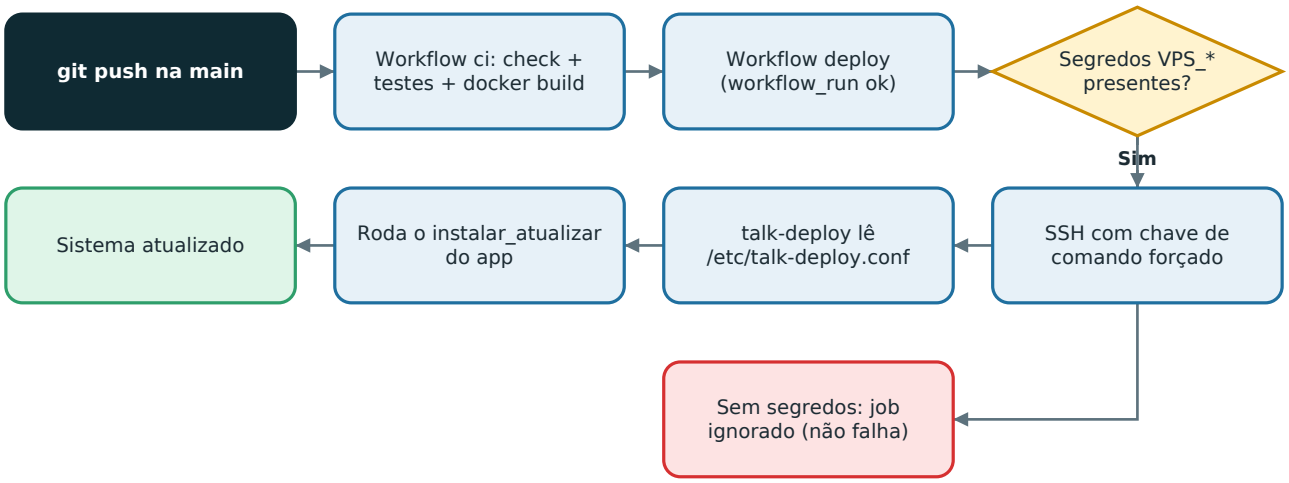


Figura 10.3 · Pipeline. A chave só executa o `talk-deploy` (PORTA, SUBDOMINIO e REPO ficam no servidor).

- Segredos do repositório: `VPS_HOST`, `VPS_USER`, `VPS_SSH_KEY`, `VPS_KNOWN_HOSTS`; variável opcional `VPS_DEPLOY_CMD`.
- `StrictHostKeyChecking=yes` com `known_hosts` fixado; `concurrency` evita deploys simultâneos.
- Detalhes em `docs/DEPLOY_AUTOMATICO.md`.

10.6 Backup e restauração

- **Banco**: `pg_dump` diário (ou PITR no provedor). **Chaves**: `DATA_KEY` e `JWT_SECRET` (de `data/config.env`) em cofre; sem a `DATA_KEY` os dados cifrados são irrecuperáveis.
- Restaurar: recriar o banco, `psql -f dump`, colocar o `config.env` original e iniciar.
- Verificação pós-restauração: `GET /api/audit/verify` deve responder ok.

10.7 Solução de problemas

Sintoma	Causa provável	Ação
DATA_KEY deve ter 32 bytes em base64	Chaves ausentes ou inválidas	Use o assistente; confira config.env
502 no proxy	App subindo ou parado	pm2 logs talk / docker compose logs app
self-signed certificate	PostgreSQL local com certificado autoassinado	Desmarque TLS no assistente ou PGSSL=false
password authentication failed	Senha do usuário do banco diferente da URL	ALTER USER … PASSWORD; teste a URL
Código de instalação inválido	Reinício regenerou o código (versões antigas)	cat data/setup-code.txt
WebSocket não conecta	Nginx sem Upgrade/Connection	Rode deploy/vps-fix-proxy.sh
Aviso de proxy HTTPS	Sem X-Forwarded-Proto	Idem
Usuário travado	8 senhas erradas	Usuários → Destravar
Tela antiga após atualizar	Cache do service worker	Aumente talk-m3d-vNN; Ctrl+F5
Vídeo sem imagem	Sem HTTPS ou sem permissão	HTTPS e permissões do navegador

11 Testes, qualidade e CI/CD

O que é verificado e como rodar

```
npm install
npm run check     # sintaxe de src, scripts, public e tests
npm test          # node --test (sem banco: lógica pura)
```

Arquivo de teste	Cobre
`assistente.test.js`	Assistente só indica funções do perfil
`billing.test.js`	Planos, tokens, custos, acesso, embaixadores
`boot.test.js`	config.env, chaves, código, verificações do boot
`caso.test.js`	Compilação do caso e pedidos à IA
`consulta.test.js`	Minutagem, biblioteca por prioridade, alertas
`crypto.test.js`	AES-GCM, tokens de convidado
`deploy.test.js`	Arquivos de implantação (compose, k8s, scripts)
`familia.test.js`	Familiograma e ecomapa
`i18n.test.js`	Regras da tradução de interface
`logic.test.js`	Estado de receitas e rotinas
`policy.test.js`	Política de senha e cadastro
`presence.test.js`	Presença e estados
`schema.test.js`	Migrações em sequência; toda tabela usada no código existe em alguma migração

Verificações fora dos testes unitários

- **E2E de interface**: scripts Playwright mantidos à parte do pacote de produção, sobre a versão de demonstração com dados simulados, cobrem desktop, menu, agenda, consulta, caso, família, créditos e instalador.
- **Roteiro de homologação** (banco e Docker reais): `/api/audit/verify` = ok; login de cada perfil; duas abas no chat; conflito de agenda; vídeo entre dois navegadores; transcrição e failsafe (`/api/ai/health`); documento verificável; compra de plano; bloqueio por teste vencido.

> **Atenção:** Os testes unitários não exercitam o servidor contra um PostgreSQL real. Antes de produção, execute o roteiro de homologação numa instalação de teste.

CI (`.github/workflows/ci.yml`)

```
on: push, pull_request
job verificar: checkout → Node 22 → npm install → npm run check → npm test
job docker:    checkout → docker build -t talk-m3d .
```

Qualidade de código

- ES modules; uma responsabilidade por arquivo; regras puras em `*Logic.js` com testes.
- SQL sempre parametrizado (`$1, $2…`); nunca concatenar entrada do usuário.
- Erros de cliente com `statusCode` 4xx e mensagem em português; 5xx genérico.
- Toda ação sensível chama `audit()`.

12 Guia de desenvolvimento

Receitas para as mudanças mais comuns

12.1 Ambiente local

```
git clone https://github.com/LACibermedicina/talk && cd talk
npm install                       # postinstall copia o SDK Agora
export DATABASE_URL=postgresql://user:pw@127.0.0.1:5432/talk
export PGSSL=false COOKIE_SECURE=false
export APP_ORIGIN=http://localhost:3000
node src/start.js                 # sem JWT_SECRET/DATA_KEY abre o instalador
# ou: TEST_MODE=true ... para contas de teste e acesso rápido
```

12.2 Adicionar uma rota

- Crie ou edite `src/routes/<dominio>.js` exportando `default async function (app)`; se for módulo novo, inclua o nome na lista de `src/app.js`.
- Escolha o acesso: `{ preHandler: app.auth }` ou `app.role(...)`; para dado de paciente, chame canSee/canSeeFull.
- Valide com Zod; use `q()` com parâmetros; cifre colunas sensíveis com `enc()`; chame `audit()`.
- Se consome IA e deve custar créditos, adicione a rota em FEATURE_ROUTES e a funcionalidade em FEATURES (`billingLogic.js`).
- Escreva teste para a lógica pura e rode `npm run check && npm test`.

12.3 Adicionar uma migração

- Crie `db/014_nome.sql` (próximo número, sem pular). É aplicada em transação ao iniciar (AUTO_MIGRATE) ou pelo passo do assistente.
- Prefira `create table if not exists` e `add column if not exists`; nunca edite migrações já aplicadas.
- `tests/schema.test.js` garante a sequência e que toda tabela usada no código existe.

12.4 Novo perfil, idioma ou item de desktop

- **Perfil**: adicione em `src/roles.js` (grupos), no CHECK de `users.role` por migração, em DOCS se emitir documentos e nos itens de dock em `public/dash.js/app.js`.
- **Idioma**: acrescente a coluna no dicionário `public/i18n-dict.js` e à lista de idiomas; textos faltantes são traduzidos por IA e cacheados.
- **Item de desktop**: registre em HOME_APPS/dock e respeite dHidden (configuração do usuário).

12.5 Fluxo de contribuição

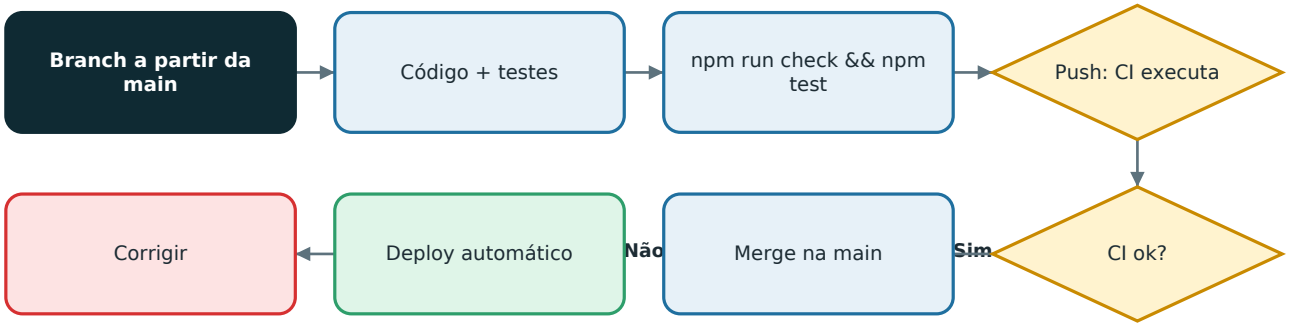


Figura 12.1 · Do código ao servidor.

13 Limites conhecidos e glossário

O que ainda não está coberto

- Assinatura de receita é **eletrônica simples** (hash + QR), não ICP-Brasil.
- E-mail não é verificado no cadastro; não há push (somente SMS e notificações na tela).
- **Uma instância** por ambiente; presença e WebSocket em memória.
- A IA é apoio à decisão e exige revisão; tradução (principalmente guarani) exige revisão humana; interações medicamentosas usam lista curta.
- O servidor de instalação, as rotas de créditos e credenciais e a migração 013 foram verificados por testes unitários e sintaxe, mas **ainda precisam do roteiro de homologação** em banco e Docker reais.
- Anotações do desktop ficam no navegador do usuário.

Glossário técnico

Termo	Significado
RBAC	Controle de acesso por perfil
Vínculo	Linha em care_team ligando paciente e profissional
DATA_KEY	Chave AES-256-GCM que cifra dados em repouso
PWA	Aplicativo web instalável com service worker
Failsafe	Troca automática de provedor de IA em caso de falha
Ledger	Razão de créditos: toda entrada e saída
Advisory lock	Trava de transação do PostgreSQL usada na cadeia de auditoria
Loopback	Endereço da própria máquina (127.0.0.1): conexão sem TLS
PM2	Gerenciador de processos Node usado no VPS
Comando forçado (SSH)	Chave que só pode executar um comando fixo (talk-deploy)